



VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
PROGRAMŲ SISTEMŲ STUDIJŲ PROGRAMA

Kursinis darbas

**Patalpos žemėlapis sudarymas naudojant
TurtleBot Burger robotą**
Creating a room map using the TurtleBot Burger robot

Gvidas Bačianskas

Denisas Savickis

Darbo vadovas : prof. dr. Virginijus Marcinkevičius

Vilnius
2026

Turinys

Įvadas	4
1. Literatūros apžvalga	6
1.1. Autonominių robotų navigacijos technologijos	6
1.1.1. Sistemos architektūra	6
1.1.2. Jutikliai aplinkos suvokimui	7
1.1.3. Judėjimo sistema ir odometrija	7
1.2. Robotikos simuliacija	7
1.2.1. ROS ir Gazebo aplinka	8
1.2.2. Simuliacijos ir realybės atotrūkis	8
1.3. Patalpų žemėlapių sudarymo metodai	8
1.3.1. Dalelių filtro metodai	9
1.3.2. Grafų optimizavimo metodai	9
1.4. Autonominio tyrinėjimo algoritmai	10
1.4.1. Ribomis paremtas metodas	10
1.4.2. Grafų paieškos metodai	10
1.5. Algoritmų vertinimo metrikos	11
2. Fizinio roboto konstrukcija ir valdymas	12
2.1. Fizinio roboto sistemos architektūra	12
2.1.1. Variklių konfigravimas	14
2.1.2. Aplinkos jutiklių integracija	15
2.2. Žemo lygio valdymo programinės įrangos kūrimas	16
2.2.1. Programavimo aplinka ir naudojami įrankiai	17
2.2.2. Programavimo kalbos ir kodo struktūros specifi­ka	17
2.2.3. Duomenų perdavimo protokolai ir valdymo logikos realizacija	18
2.3. Aukšto lygio sistemos integravimas	20
2.3.1. Operacinės sistemos paruošimas	20
2.3.2. Programiniai moduliai	21
3. Programinė architektūra	23
3.1. Sistemos struktūra	23
3.2. Architektūros diagrama	23
3.3. Pagrindinės klasės	24
3.3.1. ExplorationRunner	24
3.3.2. ExplorationStrategy	24
3.3.3. BaseNavigator	24
3.4. Duomenų struktūros	25
3.5. Tyrinėjimo ciklas	25
4. Simuliacinės aplinkos realizacija	26
4.1. Roboto modelio pritaikymas	26
4.2. Patalpos modelio kūrimas	26
4.2.1. Simuliacinė patalpa	26
5. Autonominio tyrinėjimo algoritmų realizacija	28
5.1. Ribomis paremto algoritmo realizacija	28
5.1.1. Ribų aptikimas	30

5.1.2.	Ribos reprezentacinio taško parinkimas	31
5.1.3.	Nepasiekiamų tikslų valdymas	31
5.2.	BFS algoritmo realizacija	31
5.2.1.	Maršruto planavimas	33
5.2.2.	Trajektorijos sekimas	34
5.2.3.	Saugumo mechanizmai	34
Rezultatai ir išvados		36
	Rezultatai fizinėje aplinkoje	36
	Rezultatai simuliacinėje aplinkoje	37
	Išvados	39

Įvadas

Autonominė robotika yra viena sparčiausiai besivystančių technologijų sričių, plačiai taikoma logistikos, paslaugų ir mokslinių tyrimų srityse. Autonominių mobiliųjų robotų veikimas patalpų aplinkoje reikalauja gebėjimo suvokti aplinką, planuoti judėjimą ir priimti sprendimus be tiesioginio žmogaus įsikišimo. Vienas iš esminių šio proceso elementų yra aplinkos žemėlapių sudarymas, kuris kartu su roboto lokalizacija sudaro pagrindą tolimesnei navigacijai ir tyrinėjimui. Šie gebėjimai yra būtini siekiant, kad robotas galėtų veikti savarankiškai iš anksto nežinomoje aplinkoje.

Tačiau vien tik aplinkos žemėlapių sudarymas nėra pakankamas visiškam roboto autonomiškumui užtikrinti. Robotas taip pat turi savarankiškai priimti sprendimus, kurių aplinkos dalį tyrinėti toliau, vengdamas kliūčių ir užtikrindamas saugų judėjimą. Skirtingos autonominio tyrinėjimo strategijos gali lemti reikšmingus skirtumus žemėlapių sudarymo tikslume bei užduoties atlikimo trukmėje. Todėl autonominio tyrinėjimo algoritmo pasirinkimas ir jo realizavimas turi tiesioginę įtaką visos sistemos veikimo rezultatams.

Šiame darbe pagrindinis dėmesys skiriamas ne atskiram žemėlapių sudarymo metodui, bet visam autonominio patalpų tyrinėjimo procesui – nuo roboto paruošimo ir konfigūravimo iki autonominio tyrinėjimo algoritmo realizavimo. Darbo esmė – suprasti ir įgyvendinti roboto sprendimų priėmimo logiką, leidžiančią jam savarankiškai tirti nežinomą aplinką.

Darbas atliekamas dviejų autorių, kurie nagrinėja tą pačią autonominio patalpų kartografovimo problemą skirtingose aplinkose – naudojant fizinį robotą ir simuliacinį modelį. Toks požiūris leidžia analizuoti ir palyginti autonominio tyrinėjimo algoritmų realizavimą, išlaikant bendrą darbo kryptį ir tikslą. Skirtingos realizavimo aplinkos suteikia galimybę įvertinti sprendimų pritaikomumą ir elgseną praktinėse situacijose. Tokiu būdu darbas leidžia įgyti praktinės patirties kuriant ir derinant autonomines robotines sistemas.

Šis kursinis darbas orientuotas į praktinį mokymąsi ir proceso suvokimą, todėl didelis dėmesys skiriamas savarankiškam algoritmų realizavimui bei sistemos suderinimui. Autonominio tyrinėjimo algoritmai buvo kuriami ir adaptuojami siekiant užtikrinti stabilų roboto veikimą patalpų aplinkoje.

Darbo tikslas – išanalizuoti technologijas ir algoritmus naudojamus robotų autonominiame išžvalgyme, realizuoti pasirinktus algoritmus naudojant TurtleBot burger robotą, palyginti jų efektyvumą simuliacinėje ir realioje aplinkoje.

Remiantis sudarytu darbo planu, keliama šie uždaviniai:

1. Atlikti literatūros analizę apie mobiliųjų robotų navigaciją ir patalpų žemėlapių sudarymo algoritmus.
2. Sukonstruoti TurtleBot Burger robotą ir išmokti jį valdyti bei programuoti.
3. Realizuoti TurtleBot Burger simuliacinę aplinką, išmokti jį valdyti, programuoti bei surinkti duomenis.

4. Realizuoti pasirinktus autonominio tyrinėjimo algoritmus.
5. Atlikti eksperimentinius sistemos bandymus, renkant duomenis apie žemėlapių sudarymo laiką ir tikslumą.
6. Palyginti fizinio roboto ir simuliacinio modelio veikimo rezultatus bei apibendrinti algoritmų efektyvumą skirtingose aplinkose.

1. Literatūros apžvalga

Kuriant autonominius robotus, įprasta inžinerinė praktika yra pradėti nuo algoritmų modeliavimo virtualioje erdvėje. Tai leidžia saugiai, greitai ir be kaštų patikrinti navigacijos principus. Tačiau perėjimas nuo simuliacinio prie fizinio modelio dažnai atskleidžia naujas problemas, kurios simuliacijoje neegzistavo. Dėl šios priežasties literatūros analizė yra svarbi siekiant suprasti, kaip teoriniai sprendimai pritaikomi praktinėse sistemose.

Šiame skyriuje apžvelgiama robotų kūrimo metodika, „TurtleBot3“ platformos ypatybės bei patalpų kartografavimo (SLAM) ir autonominio tyrinėjimo technologijos. Ši apžvalga sudaro teorinį pagrindą tolimesniuose skyriuose aprašomam autonominio roboto sistemos realizavimui.

1.1. Autonominių robotų navigacijos technologijos

Autonominio roboto navigacijos sistema yra sudėtinga ir sudaryta iš kelių tarpusavyje sąveikaujančių technologinių komponentų. Siekiant užtikrinti patikimą ir lankstų roboto veikimą, šios sistemos dažniausiai projektuojamos sluoksniuotos architektūros principu, atskiriant žemo lygio valdymą nuo aukšto lygio sprendimų priėmimo.

Šiame darbe naudojama „TurtleBot3 Burger“ platforma [AS19; ROBe], kuri yra tipinis modulinės robotinės sistemos pavyzdys. Toliau šiame poskyryje apžvelgiami pagrindiniai technologiniai komponentai, kurie naudojami roboto navigacijos uždaviniams spręsti, bei jų vaidmuo bendroje sistemos struktūroje.

1.1.1. Sistemos architektūra

Autonominių robotų navigacijos sistemos dažniausiai projektuojamos dviejų lygių architektūros principu, atskiriant žemo lygio valdymą nuo aukšto lygio skaičiavimų. Žemajame lygyje naudojamas mikrovaldiklis, atsakingas už tiesioginę sąveiką su fizineis komponentais – variklių valdymą, jutiklių duomenų nuskaitymą ir realaus laiko operacijas. Aukštajame lygyje veikia pagrindinis kompiuteris, kuris vykdo sudėtingesnius algoritmus, tokius kaip SLAM ir autonominis tyrinėjimas.

„TurtleBot3“ platformoje šią architektūrą realizuoja „OpenCR“ mikrovaldiklis [ROBc] ir „Raspberry Pi“ vienplokštis kompiuteris [Pi]. Mikrovaldiklis veikia kaip tarpinė grandis tarp aukšto lygio skaičiavimo sistemos ir fizinės roboto įrangos, užtikrindamas deterministinį veikimą realiuoju laiku. „Raspberry Pi“ kompiuteris suteikia pakankamai procesoriaus galios ir operatyviosios atminties navigacijos algoritmams vykdyti, o taip pat leidžia diegti pilnavertes operacines sistemas ir programinius karkasus, tokius kaip ROS.

Toks architektūrinis sprendimas leidžia efektyviai paskirstyti skaičiavimo apkrovą – žemo lygio valdymo funkcijos vykdomos nepriklausomai nuo pagrindinio kompiuterio apkrovos, taip padidinant visos sistemos patikimumą ir palengvinant programinės įrangos plėtrą.

1.1.2. Jutikliai aplinkos suvokimui

Aplinkos suvokimas yra viena svarbiausių autonominio mobiliojo roboto funkcijų, nuo kurios tiesiogiai priklauso navigacijos ir autonominio tyrinėjimo algoritmų veikimas. Siekiant sudaryti aplinkos žemėlapią ir aptikti kliūtis, mobiliuosiuose robotuose naudojami įvairūs jutikliai, tokie kaip ultragarsiniai davikliai, kameros ar lazeriniai atstumų matuokliai.

Literatūroje pabrėžiama, kad patalpų navigacijos uždaviniams dažniausiai pasirenkami lazeriniai atstumų matuokliai (LiDAR), nes jie leidžia tiksliai ir greitai nustatyti atstumą iki aplinkinių objektų. LiDAR jutikliai skleidžia lazerio spindulį ir, matuodami jo atspindžio laiką, apskaičiuoja atstumą iki kliūtis, taip sudarydami dvimatį aplinkos vaizdą.

Dėl plataus matymo lauko ir pakankamo atnaujinimo dažnio lazeriniai jutikliai yra ypač tinkami autonomiam patalpų tyrinėjimui. Jie suteikia roboto navigacijos sistemai nuoseklius ir struktūrizuotus duomenis, kurie gali būti tiesiogiai naudojami žemėlapių sudarymo ir sprendimų priėmimo algoritmuose. Be to, LiDAR jutiklių duomenys yra mažiau jautrūs apšvietimo sąlygoms nei vaizdo kameros, todėl jie laikomi patikimu sprendimu vidaus aplinkose [ROBB].

1.1.3. Judėjimo sistema ir odometrija

Mobiliųjų robotų judėjimas realizuojamas naudojant elektromechanines pavaras, kurios leidžia robotui keisti padėtį erdvėje ir vykdyti planuotą trajektoriją. Vidaus patalpų navigacijoje dažniausiai naudojami ratais varomi robotai, pasižymintys paprasta mechanine konstrukcija ir pakankamu manevringumu.

Vienas iš plačiai taikomų sprendimų yra diferencinio valdymo schema, kai robotas turi du nepriklausomai valdomus varomuosius ratus. Keičiant ratų sukimosi greitį, robotas gali judėti tiesiai, sukis vietoje arba važiuoti lenkta trajektorija. Dėl savo paprastumo ir efektyvumo ši valdymo schema naudojama tiek edukacinėse, tiek tyrimų platformose.

Svarbus judėjimo sistemos komponentas yra odometrija – roboto nueito kelio apskaičiavimas pagal ratų apsisukimų skaičių. Dauguma mobiliųjų robotų naudoja variklius su integruotais koduotojais (angl. Encoder), kurie leidžia matuoti ratų poziciją ir greitį [ROBf]. Odometrijos duomenys yra būtini navigacijos sistemai, nes suteikia informaciją apie roboto judėjimą tarp jutiklių matavimų. Tačiau dėl ratų praslydimo ir mechaninių paklaidų odometrija ilgainiui kaupia klaidas, todėl ją būtina derinti su kitais lokalizacijos metodais, tokiais kaip SLAM.

1.2. Robotikos simuliacija

Šiuolaikinių autonominių robotų kūrimo procese simuliacija tapo neatsiejama inžinerinio ciklo dalimi. Prieš diegiant valdymo algoritmus į fizinę įrangą, jie yra testuojami virtualiose aplinkose, kas leidžia ankstyvose kūrimo stadijose įvertinti algoritmo veikimą ir sumažinti galimų klaidų skaičių. Simuliacijos naudojimas turi akivaizdžių privalumų – ji leidžia greitai patikrinti navigacijos logiką, nerizikuojant sugadinti įrangos susidūrimų metu, bei atlikti daug bandymų per trumpą laiką.

1.2.1. ROS ir Gazebo aplinka

Robotikos tyrimuose ir edukacinėse platformose dažniausiai naudojamas **ROS** (angl. Robot Operating System) ir „**Gazebo**” simulatoriaus derinys. ROS yra atviro kodo programinis karkasas, suteikiantis standartizuotas bibliotekas, komunikacijos protokolus ir įrankius robotų aplikacijų kūrimui [QCG⁺09]. „Gazebo” yra fizikos simulatorius, leidžiantis sukurti tikslų robotų kinematinį modelį, imituoti jutiklių duomenis ir modeliuoti aplinkos fiziką – gravitaciją, trintį bei susidūrimus.

Šių įrankių derinys leidžia kurti ir testuoti navigacijos algoritmus virtualioje aplinkoje prieš juos diegiant į fizinį robotą. „TurtleBot3” platforma turi oficialius „Gazebo” modelius, kurie tiksliai atitinka fizinio roboto parametrus, todėl algoritmai, sukurti simuliacijoje, gali būti tiesiogiai perkelti į realų robotą.

1.2.2. Simuliacijos ir realybės atotrūkis

Pereinant nuo virtualaus modelio prie fizinio robotų, dažnai susiduriama su reiškiniu, literatūroje vadinamu „realybės atotrūkiu” (angl. Sim-to-Real Gap). Šis reiškinys apibūdina situaciją, kai algoritmai, demonstruojantys gerus rezultatus simuliacijoje, fizinėje aplinkoje veikia ne taip efektyviai arba reikalauja papildomo pritaikymo.

Pagrindinės šio neatitikimo priežastys yra susijusios su fizinių procesų ir techninės įrangos netobulumais realybėje:

1. **Fizikinė sąveika ir trintis:** Simulatoriai dažnai naudoja supaprastintus trinties modelius. Realiojo pasaulyje diferencinio valdymo robotai yra jautrūs ratų praslydimui, kas lemia odometrijos paklaidas [AS19].
2. **Jutiklių triukšmas:** Simuliuotas LiDAR jutiklis grąžina idealius atstumo matavimus. Tuo tarpu realus jutiklis yra veikiamas aplinkos triukšmo, atspindžių nuo blizgių paviršių bei matavimo paklaidų, kurias būtina filtruoti programiškai.
3. **Vykdomo laikas ir resursai:** Simuliacijoje kompiuterio resursai dažnai nėra griežtai ribojami, o laikas gali būti stabdomas. Realus mikrovaldiklis turi ribotą skaičiavimo galią ir privalo apdoroti duomenis realiu laiku.

Šie veiksniai lemia, kad net ir gerai simuliacijoje veikiantys algoritmai realiose sistemose dažnai reikalauja papildomos adaptacijos. Dėl šios priežasties šiame darbe algoritmai testuojami tiek simuliacinėje, tiek realioje aplinkoje, siekiant įvertinti jų praktinį pritaikomumą.

1.3. Patalpų žemėlapių sudarymo metodai

Kad mobilusis robotas galėtų savarankiškai judėti nežinomoje patalpų aplinkoje, jis turi gebėti sukurti vidinį aplinkos modelį ir tuo pačiu metu nustatyti savo padėtį tame modelyje. Ši problema literatūroje vadinama SLAM (angl. Simultaneous Localization and Mapping) – viena-
laikė lokalizacija ir žemėlapių sudarymas.

SLAM algoritmai dažniausiai naudoja užimtumo tinklo (angl. Occupancy Grid) reprezentaciją, kurioje aplinka suskaidoma į reguliaraus dydžio langelius. Kiekvienas langelis gali būti priskirtas vienai iš trijų būsenų: laisva, užimta arba nežinoma. Toks aplinkos atvaizdavimas leidžia robotui sistemingai kaupti informaciją ir nuolat atnaujinti žemėlapių judėjimo metu [CC07]. Užimtumo tinklai sudaro pagrindą tiek navigacijos, tiek autonominio tyrinėjimo algoritmams, nes leidžia aiškiai identifikuoti neištirtas aplinkos zonas.

Toliau šiame poskyryje apžvelgiamos dvi pagrindinės SLAM algoritmų grupės, kurios skiriasi matematiniu pagrindu ir taikymo ypatybėmis.

1.3.1. Dalelių filtro metodai

Viena iš klasikinių SLAM problemų sprendimo strategijų remiasi dalelių filtru (angl. Particle Filter), kuris yra Monte Karlo lokalizacijos metodo išplėtimas. Šiuo metodu roboto būsena ir žemėlapis yra aprašomi tikimybiškai, naudojant aibę svertinių dalelių, kurios reprezentuoja galimas roboto trajektorijas.

Vienas žinomiausių šios grupės algoritmų yra GMapping, paremtas Rao-Blackwellized dalelių filtru. Algoritmas kiekvienai dalelei priskiria atskirą žemėlapio hipotezę ir atnaujina dalelių svorius pagal jutiklių matavimų tikimybę. Tokiu būdu algoritmas vienu metu sprendžia ir lokalizacijos, ir žemėlapio sudarymo uždavinius.

Dalelių filtro metodų privalumas yra gebėjimas apdoroti netiesinius modelius ir daugiapikes tikimybių pasiskirstymus. Tačiau šie metodai reikalauja pakankamo dalelių skaičiaus tiksliai žemėlapiui sudaryti, o tai didina skaičiavimo apkrovą. Be to, dalelių filtro SLAM algoritmai yra jautrūs odometrijos tikslumui – didelės odometrijos paklaidos gali lemti neteisingą dalelių pasiskirstymą ir žemėlapio iškraipymus.

1.3.2. Grafų optimizavimo metodai

Kita plačiai taikoma SLAM algoritmų grupė remiasi grafų optimizavimu (angl. Graph-based SLAM arba Pose Graph SLAM). Šiuose methoduose roboto trajektorija vaizduojama kaip grafas, kurio viršūnės atitinka roboto pozicijas skirtingais laiko momentais, o briaunos – apribojimus tarp šių pozicijų, gautus iš odometrijos ar jutiklių matavimų.

Grafų optimizavimo metodų pagrindinis privalumas yra gebėjimas efektyviai spręsti kilpų uždarymo (angl. Loop Closure) problemą. Kai robotas grįžta į anksčiau lankytą vietą, algoritmas aptinka šį įvykį ir optimizuoja visą pozicijų grafą, koreguodamas sukauptas paklaidas. Tai leidžia sudaryti nuoseklius ir tikslius didelių erdvių žemėlapius.

ROS2 aplinkoje plačiai naudojamas „SLAM Toolbox“ paketas, kuris realizuoja grafų optimizavimo principus. Šis įrankis pasižymi gebėjimu efektyviai apdoroti jutiklių ir odometrijos duomenis bei sumažinti klaidų kaupimąsi [MJ21]. Kitas žinomas šios grupės atstovas yra „Google Cartographer“, kuris naudoja subžemėlapių (angl. Submaps) koncepciją ir palaiko tiek 2D, tiek 3D SLAM režimus.

Lyginant su dalelių filtro metodais, grafų optimizavimo algoritmai dažnai pasiekia didesnį žemėlapio tikslumą ir geriau susitvarko su didelėmis aplinkomis. Tačiau jie reikalauja daugiau at-

minties grafo struktūrai saugoti ir gali būti skaičiavimų požiūriu intensyvu optimizuojant didelius grafus.

1.4. Autonominio tyrinėjimo algoritmai

Norint, kad mobilusis robotas galėtų savarankiškai sudaryti patalpų žemėlapi, nepakanka vien aplinkos suvokimo ir žemėlapio sudarymo metodų. Robotas taip pat turi gebėti priimti sprendimus, nurodančius, kuri aplinkos dalis turėtų būti tirama toliau. Šie sprendimai priimami naudojant autonominio tyrinėjimo algoritmus, kurie analizuoja turimą žemėlapi ir parenka tolimesnius judėjimo tikslus.

Literatūroje siūloma įvairių autonominio tyrinėjimo strategijų, kurios skiriasi sudėtingumu, skaičiavimo kaštais ir elgsena realiose sistemose. Toliau šiame skyriuje apžvelgiami du plačiai naudojami metodai: ribomis paremtas tyrinėjimas[Yam97] ir grafų paieškos metodai.

1.4.1. Ribomis paremtas metodas

Vienas iš labiausiai paplitusių autonominio tyrinėjimo metodų yra ribomis paremtas (angl. Frontier-Based) algoritmas, pirmą kartą pasiūlytas B. Yamauchi 1997 metais [Yam97]. Šio metodo pagrindinė idėja – identifikuoti ribas, kurios skiria jau ištirtą aplinkos dalį nuo dar nežinomos erdvės.

Algoritmas analizuoja užimtumo tinklą ir ieško langelių, kurie yra pažymėti kaip laisvi, bet ribojasi su nežinomomis sritimis. Šie langeliai sugrupuojami į regionus, vadinamus frontais, kurie laikomi potencialiais tyrinėjimo tikslais. Pasirinkęs vieną iš frontų, robotas planuoja maršrutą link jo ir taip palaipsniui plečia žinomą aplinkos dalį.

Fronto pasirinkimo strategija turi įtakos tyrinėjimo efektyvumui. Dažniausiai naudojamos dvi strategijos: artimiausio fronto (angl. Closest Frontier), kuri minimizuoja nuvažiuotą atstumą, ir didžiausio fronto (angl. Largest Frontier), kuri pirmenybę teikia didelėms neištirtoms zonoms. Ribomis paremti algoritmai yra vertinami dėl savo paprastumo ir sistematiško padengimo, tačiau gali reikalauti dažnos viso žemėlapio analizės, kas didina skaičiavimo apkrovą didesnėse aplinkose.

1.4.2. Grafų paieškos metodai

Kita plačiai taikoma autonominio tyrinėjimo metodų grupė remiasi grafų teorija ir paieškos algoritmais. Šie metodai dažnai naudojami tiek maršruto planavimui, tiek artimiausio tinkamo tyrinėjimo tikslo paieškai. Vienas iš klasikinių pavyzdžių yra paieška pločiu (angl. Breadth-First Search – BFS), detaliai aprašyta algoritmų literatūroje [CLR⁺22].

Grafų paieškos metoduose užimtumo tinklas gali būti interpretuojamas kaip grafas, kurio viršūnės atitinka laisvas aplinkos ląsteles, o briaunos – galimus judėjimo perėjimus tarp jų. BFS algoritmas pradeda paiešką nuo pradinės roboto padėties ir plečia paieškos sritį sluoksniais, tikrindamas artimiausius kaimyninius mazgus.

Pagrindinis BFS metodo privalumas yra jo pilnumas – jei egzistuoja pasiekiamas tikslas, algoritmas garantuoja, kad jis bus rastas. Dėl šios savybės grafų paieškos metodai dažnai naudo-

jami autonominio tyrinėjimo uždaviniuose, kuriuose svarbu užtikrinti, kad robotas sistemingai ir nuosekliai ištirtų visą aplinką.

1.5. Algoritmų vertinimo metrikos

Siekiant objektyviai palyginti autonominio tyrinėjimo algoritmus, būtina apibrėžti kiekybinės vertinimo metrikas. Kadangi abu lyginami algoritmai naudoja tą patį SLAM įrankį žemėlapiui sudaryti, vertinimo metrikos orientuotos į tyrinėjimo proceso efektyvumą.

Išžvalgymo laikas – laikas nuo tyrinėjimo pradžios iki pasiekto padengimo lygio (dažniausiai 95% prieinamos erdvės). Ši metrika tiesiogiai parodo algoritmo greitį ir yra viena svarbiausių praktinėse taikomose srityse, kur robotas turi greitai ištirti nežinomą aplinką.

Nuvažiutas atstumas – bendra roboto trajektorijos ilgis tyrinėjimo metu. Mažesnis nuvažiuotas atstumas rodo efektyvesnį maršruto planavimą ir mažesnes energijos sąnaudas. Ši metrika ypač svarbi mobiliems robotams su ribota baterijos talpa.

Padengimo pilnumas – procentinė išžvalgytos erdvės dalis nuo visos prieinamos aplinkos ploto. Ši metrika parodo, ar algoritmas sugeba sistemingai ištirti visą aplinką, nepaliekant neištirtų zonų.

Šios trys metrikos bus naudojamos eksperimentinėje dalyje, lyginant Frontier-based ir BFS algoritmų veikimą simuliacinėje ir realioje aplinkoje.

2. Fizinio roboto konstrukcija ir valdymas

Šiame skyriuje aprašomas autonominio roboto sistemos realizavimo procesas, kuris buvo vykdomas naudojant „TurtleBot3 Burger“ platformą. Skirtingai nuo simuliacinių tyrimų, kuriuose visi sistemos komponentai veikia virtualioje aplinkoje, fizinio roboto realizacija reikalauja nuoseklaus techninės ir programinės įrangos paruošimo bei jų tarpusavio suderinimo.

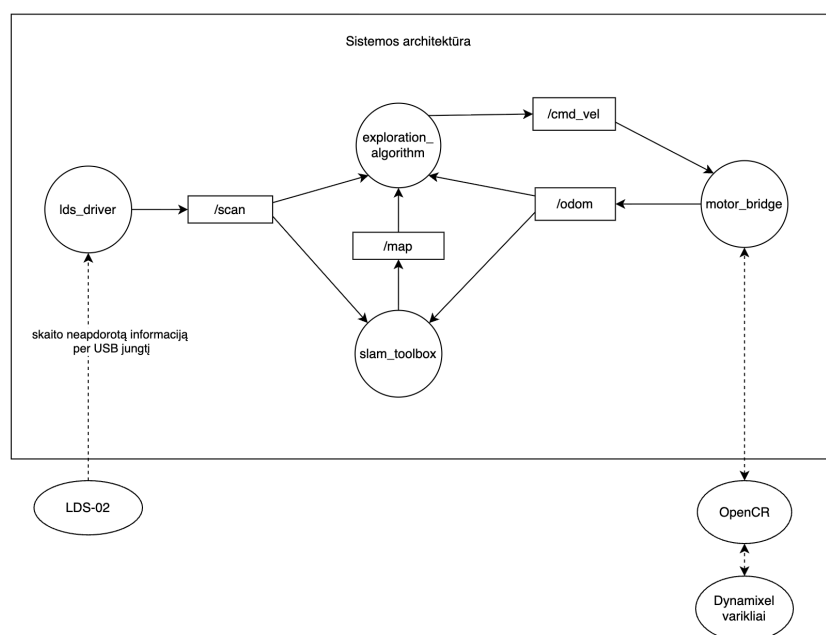
Darbo metu robotas buvo surinktas laikantis gamintojo pateiktų instrukcijų, o vėliau paruoštas autonominiam veikimui patalpų aplinkoje. Šis procesas apėmė mechaninės dalies surinkimą, variklių ir jutiklių paruošimą, žemo lygio valdymo programinės įrangos kūrimą bei aukšto lygio navigacijos ir tyrinėjimo algoritmų integravimą.

Toliau šiame skyriuje nuosekliai aprašoma fizinio roboto sistemos architektūra, atskirų komponentų konfigūravimas ir programinės įrangos sprendimai, leidę realizuoti autonominę patalpų tyrinėjimą realioje aplinkoje.

2.1. Fizinio roboto sistemos architektūra

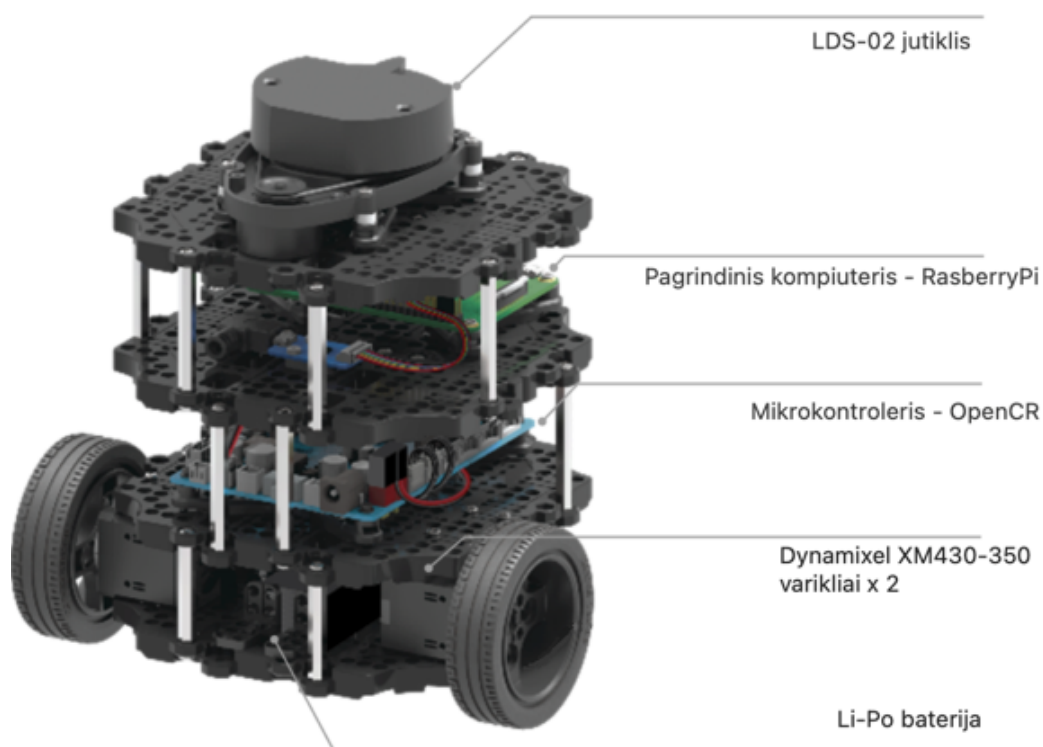
Šiame poskyryje aprašoma fizinio roboto sistemos architektūra ir techninės įrangos paruošimas, kurie sudaro pagrindą tolimesniam programinės įrangos kūrimui ir autonominio tyrinėjimo algoritmo realizavimui. Darbo metu robotas buvo surinktas laikantis gamintojo pateiktų instrukcijų, užtikrinant teisingą visų mechaninių ir elektroninių komponentų išdėstymą.

Sukurta roboto sistema sudaryta iš kelių pagrindinių komponentų: variklių, aplinkos suvokimo jutiklių, žemo lygio valdymo mikrovaldiklio ir pagrindinio skaičiavimo kompiuterio. Šie komponentai tarpusavyje sąveikauja pagal iš anksto apibrėžtą architektūrą, leidžiančią atskirti žemo lygio fizinių įrenginių valdymą nuo aukšto lygio navigacijos ir sprendimų priėmimo logikos. Architektūra apibrėžė pagrindinius ROS mazgus temas (angl. Topics), pagal kuriuos buvo kuriamas ir integruojama visa roboto sistema žr. 1 paveiksliuką.



1 pav. Fizinio roboto sistemos architektūra

Toliau šiame poskyryje aprašomi pagrindiniai fizinio roboto sistemos komponentai ir jų paruošimas autonominiam veikimui patalpų aplinkoje. Roboto struktūra iliustruojama 2 paveikslėlyje.



2 pav. TurtleBot3 Burger roboto pagrindiniai komponentai

3 paveikslėlyje pateikta viena iš patalpų, kurioje buvo atliekami autonominio roboto bandymai. Eksperimentų metu patalpos išdėstymas laikui bėgant keitėsi, siekiant išbandyti roboto navigacijos ir autonominio tyrinėjimo algoritmo veikimą įvairiomis sąlygomis.



3 pav. Eksperimentinė patalpa sukurta algoritmų testavimui

2.1.1. Variklių konfigūravimas

Vienas iš svarbiausių fizinio roboto sistemos komponentų yra judėjimo pavaros, nuo kurių tiesiogiai priklauso roboto mobilumas ir navigacijos tikslumas. Šiame darbe naudojamas „TurtleBot3 Burger“ robotas yra aprūpintas dviem „Dynamixel XM430–W350“ varikliais, kurie leidžia realizuoti diferencinio valdymo judėjimo schemą.

Kadangi „Dynamixel“ varikliai turi integruotą mikrovaldiklį ir palaiko įvairius veikimo režimus, prieš pradedant roboto programavimą buvo būtina atlikti jų pradinę konfigūraciją. Šis etapas yra svarbus siekiant užtikrinti stabilų ryšį su varikliais ir teisingą komandų vykdymą vėlesniuose sistemos darbo etapuose.

Variklių paruošimui buvo naudojama specializuota programinė įranga „Dynamixel Wizard 2.0“ [ROBa], kuri leidžia patikrinti ryšį su įrenginiais ir konfigūruoti pagrindinius jų parametrus. Prijungus variklius prie kompiuterio, buvo atliktas ryšio testas ir nustatyti identifikaciniai

numeriai, kad kiekvienas variklis galėtų būti valdomas atskirai. Tai yra būtina sąlyga diferencinio roboto judėjimui realizuoti. Prijungus variklius prie kompiuterio, buvo nustatyti šie parametrai:

1. **Identifikaciniai numeriai:** Gamykliškai visi varikliai turi tą patį ID (1), kas sukelia konfliktą. Konfigūravimo metu kairiajam varikliui buvo priskirtas ID 1, o dešiniajam – ID 2. Šis žingsnis yra esminis, nes vėliau rašant valdymo kodą, komandos siunčiamos nurodant būtent šiuos adresus.
2. **Ryšio sparta:** Patikrinus gamyklinius nustatymus, palikta numatytoji 1 Mbps (1 000 000 bps) duomenų perdavimo sparta. Testavimo metu nustatyta, kad ši sparta užtikrina stabilų duomenų srautą be paketų praradimo.

Be to, buvo pasirinktas greičio valdymo režimas, leidžiantis programiškai nustatyti ratų sukimosi greitį ir taip valdyti roboto judėjimą, siunčiant atitinkamas valdymo komandas.

Dynamixel varikliai palaiko kelis režimus:

1. srovės valdymo;
2. greičio valdymo;
3. pozicijos valdymo (0° – 360°);
4. išplėstas pozicijos valdymo;
5. pozicijos valdymo remiantis srove;
6. įtampos valdymo.

Šie valdymo režimai suteikia robotui lankstumo, kadangi galima pasirinkti kaip norima jį valdyti skirtingais panaudojimo atvejais pagal konkrečius poreikius.

2.1.2. Aplinkos jutiklių integracija

Siekiant užtikrinti roboto gebėjimą suvokti aplinką ir autonomiškai judėti patalpų erdvėje, į sistemą buvo integruotas lazerinis atstumo matavimo jutiklis. Šis jutiklis yra pagrindinis informacijos šaltinis, naudojamas kliūčių aptikimui, aplinkos žemėlapių sudarymui ir autonominio tyrinėjimo algoritmo veikimui.

Naudojamas lazerinis jutiklis generuoja dvimatę aplinkos skenavimą, kuris leidžia nustatyti atstumą iki aplinkinių objektų skirtingomis kryptimis. Tokie duomenys suteikia robotui galimybę nuolat atnaujinti aplinkos reprezentaciją ir priimti sprendimus dėl tolimesnio judėjimo. Jutiklio techninės charakteristikos ir pagrindiniai parametrai pateikiami 1 lentelėje.

1 lentelė. LDS-02 jutiklio specifikacija

Specifikacija	Reikšmė
Aptikimo atstumas	160 - 8000 mm
Matymo kampas	360°
Skenavimo dažnis	5Hz
Matavimo tikslumas	+/-10 mm
Kampinė skiriamoji geba	1°
Diskretizavimo dažnis	2,3kHz

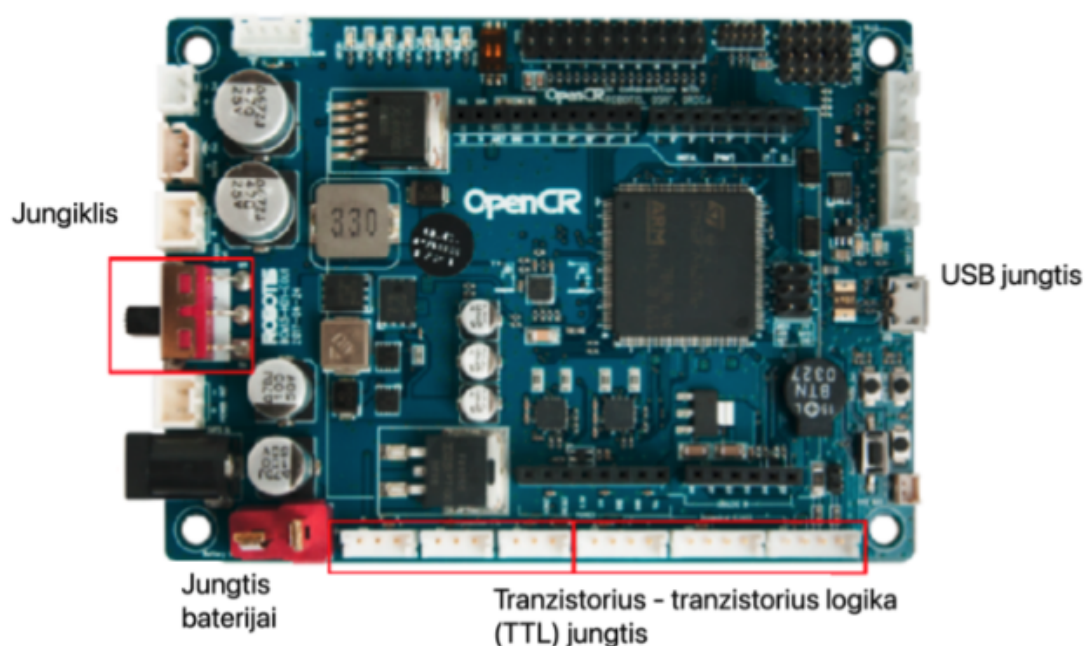
Jutiklis pritvirtintas viršutiniame roboto sluoksnyje, užtikrinant nekliudomą 360 laipsnių matymo lauką.

Programinės integracijos metu buvo užtikrintas nuoseklus jutiklio duomenų perdavimas į pagrindinį skaičiavimo kompiuterį, kuriame šie duomenys toliau apdorojami navigacijos ir tyrimo algoritmų reikmėms. Tokia integracija leidžia realiuoju laiku naudoti jutiklio informaciją aplinkos suvokimui ir roboto judėjimo planavimui.

Atlikus šiuos paruošiamuosius darbus, buvo pereita prie programinės įrangos kūrimo.

2.2. Žemo lygio valdymo programinės įrangos kūrimas

Šiame skyriuje aprašomas žemo lygio roboto valdymo programinės įrangos kūrimas, kuris buvo realizuotas naudojant „OpenCR“ mikrovaldiklį. Ši programinės įrangos dalis yra atsakinga už tiesioginį roboto fizinių komponentų valdymą ir sudaro sąsają tarp techninės įrangos ir aukšto lygio valdymo sistemos. Pagrindinės OpenCR mikrovaldiklio dalys yra pavaizduotos 4 paveikslėlyje.



4 pav. OpenCR mikrovaldiklio elementai

„OpenCR“ mikrovaldiklis pasirinktas kaip pagrindinė žemo lygio valdymo platforma dėl jo suderinamumo su „TurtleBot3“ sistema ir palaikymo ROS ekosistemoje. Jis leidžia valdyti variklius, nuskaityti jutiklių duomenis ir perduoti reikalingą informaciją pagrindiniam skaičiavimo kompiuteriui, užtikrinant patikimą ir stabilų roboto elgesį.

Toliau šiame skyriuje aprašoma naudojama programavimo aplinka, taikyti įrankiai ir pagrindiniai sprendimai, priimti kuriant žemo lygio valdymo programinę įrangą.

2.2.1. Programavimo aplinka ir naudojami įrankiai

Programos kūrimui buvo naudojama **Arduino IDE** aplinka. Nors ši platforma pirmiausia sukurta Arduino šeimos mikrovaldikliams, dėl savo lankstumo ir atvirojo kodo architektūros ji yra pilnai suderinama su projekte naudojama OpenCR plokšte. Aplinka suteikia visus būtinus įrankius pilnam kūrimo ciklui: kodo redaktorių, kompiliatorių bei įkėlimo į fizinį įrenginį sąsają [Ard].

Vienas esminių šios aplinkos privalumų – modulinė bibliotekų sistema, leidžianti efektyviai integruoti trečiųjų šalių tvarkykles. Siekiant supaprastinti darbą su „Dynamixel XM430“ varikliais, į projektą buvo įtraukta „Dynamixel2Arduino“ biblioteka.

Norint įtraukti biblioteką reikia atlikti šiuos veiksmus:

1. Atverti bibliotekų valdymo meniu (Sketch → Include Library → Manage Libraries).
2. Paieškos lauke surasti oficialią Dynamixel2Arduino biblioteką.
3. Įdiegti naujausią stabilią versiją, kuri užtikrina suderinamumą su pasirinktu ryšio protokolu.

Derinimo procesui (angl. Debugging) ypač naudingas aplinkoje integruotas serijinis monitorius (angl. Serial Monitor). Šis įrankis leido siųsti komandas, realiuoju laiku stebėti duomenų srautus tarp kompiuterio ir valdiklio, analizuoti siunčiamus paketus bei greitai identifikuoti logines klaidas programoje.

2.2.2. Programavimo kalbos ir kodo struktūros specifika

Kodo rašymui naudojamas C++ kalbos variantas, pritaikytas įterptinėms sistemoms. Ši kalba išlaiko standartinę C++ sintaksę, tačiau yra papildyta specifinėmis instrukcijomis, skirtomis tiesioginiam mikrovaldiklio valdymui. Programos architektūra remiasi specifiniu dviejų funkcijų modeliu, kuris skiriasi nuo standartinių kompiuterinių programų:

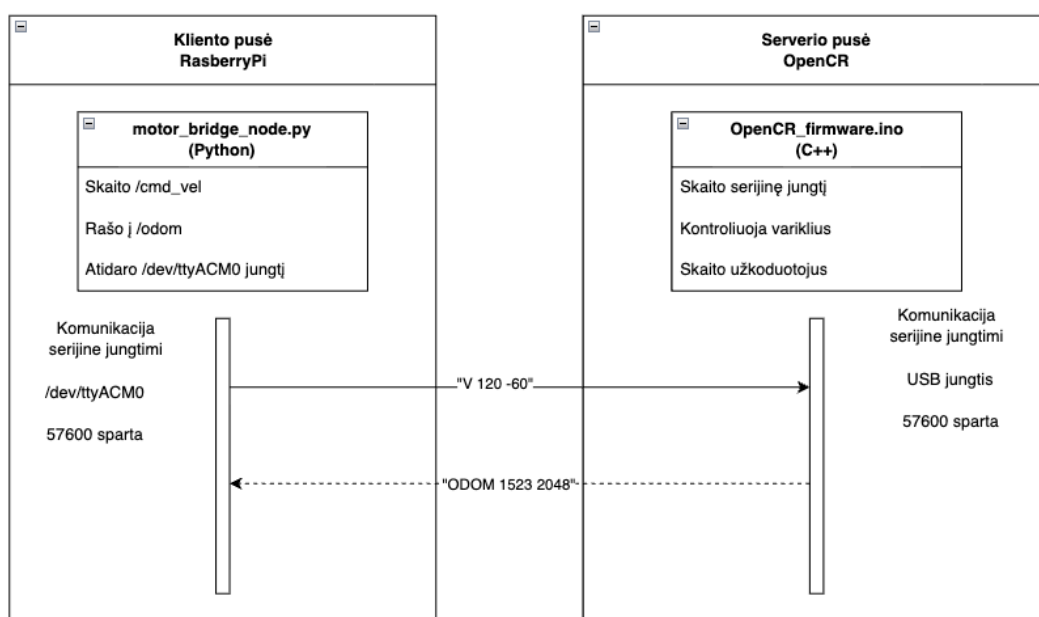
- **setup()** Ši funkcija vykdoma tik vieną kartą – sistemos paleidimo metu. Projekte ji naudojama pradinei konfigūracijai: nustatoma ryšio sparta, priskiriami kintamieji ir aktyvuojamas sukimo momentas varikliuose.
- **loop()** Tai pagrindinė funkcija, kuri pasibaigus **setup()** daliai, vykdoma begaliniu ciklu. Čia realizuota pagrindinė valdymo logika: nuolatinis komandų laukimas, jų apdorojimas ir jutiklių duomenų atnaujinimas. Tokia struktūra leidžia sistemai veikti realiu laiku.

2.2.3. Duomenų perdavimo protokolai ir valdymo logikos realizacija

Sukūrus programavimo aplinką, pagrindinis inžinerinis uždavinys buvo realizuoti patikimą dvipusį ryšį tarp Raspberry Pi ir OpenCR. Nors egzistuoja standartizuoti protokolai, šiame darbe pasirinkta kurti autorinį tekstinį (ASCII) protokolą per USB serijinę sąsają. Toks sprendimas priimtas dėl dviejų priežasčių:

1. Tekstines komandas galima lengvai stebėti ir testuoti per serijinį monitorių, nenaudojant sudėtingų dekodavimo įrankių.
2. Programuotojas turi pilną kontrolę kaip siunčiami duomenys išvengiant nereikalingo srauto (angl. overhead) būdingo universalioms bibliotekoms.

5 paveikslėlyje pateikta žemo lygio roboto valdymo programinės įrangos komunikacijos schema, paremta kliento–serverio architektūros principu. Aukšto lygio sistema, veikianti pagrindiniame skaičiavimo kompiuteryje, atlieka kliento vaidmenį ir siunčia valdymo komandas žemo lygio valdymo sistemai, realizuotai „OpenCR“ mikrovaldiklyje, kuris veikia kaip serveris.



5 pav. Kliento-serverio diagrama

Programos veikimo logika OpenCR valdiklyje apima 2 pagrindines funkcijas:

1. **Valdymo komandų apdorojimas:** Mikrovaldiklis nuolat stebi įvesties buferį. Gavus duomenų seką pradedančią simboliu V aktyvuojamas komandos vykdymo algoritmas. Ši komanda skirta tiesioginiam variklių greičio nustatymui.
 - **Protokolo formatas:** V <kairio_greitis> <dešinio_greitis>
 - **Veikimas:** Programa nuskaitymo po simbolio V einančius skaitmenis, atskirtus tarpu. Šios reikšmės konvertuojamos į sveikuosius skaičius ir perduodamos varikliams.

```

if (!COMM_SERIAL.available()) return;

String cmd = readLine();
cmd.toUpperCase();
if (cmd.length() == 0) return;

switch (cmd.charAt(0)) {
  case 'V': {
    // Format: V<left> <right>; "V120 -60"
    String args = cmd.substring(1);
    args.trim();

    int spaceIdx = args.indexOf(' ');
    if (spaceIdx > 0) {
      int v_left  = args.substring(0, spaceIdx).toInt();
      int v_right = args.substring(spaceIdx + 1).toInt();

      safeTorqueOn();
      setVelocityBoth(v_left, v_right);
    }
  } break;
}

```

Šiame fragmente OpenCR mikrovaldiklis veikia kaip serveris: jis priima tekstinę komandą per serijinę jungtį ir atitinkamai nustato variklių greičius.

2. **Odometrija:** Kad galėtume tiksliai skaičiuoti roboto poziciją erdvėje, būtina nuolat gauti grįžtamąjį ryšį iš ratų. Tam sukurtas ODOM pranešimas. Kas 20 milisekundžių (50 Hz dažniu) nuskaitomos variklių enkoderių reikšmės ir jos atspausdinamos.

Protokolo formatas: ODOM <kairio_tikai> <dešinio_tikai>

```

unsigned long now = millis();
if (now - last_odom_time >= ODOM_INTERVAL) {
  int32_t left_pos  = dxl.getPresentPosition(LEFT_ID);
  int32_t right_pos = dxl.getPresentPosition(RIGHT_ID);

  COMM_SERIAL.print("ODOM ");
  COMM_SERIAL.print(left_pos);
  COMM_SERIAL.print(" ");
  COMM_SERIAL.println(right_pos);
}

```

```

        last_odom_time = now;
    }

```

Fragmentas parodo periodinį roboto būsenos duomenų perdavimą: kas nustatytą intervalą nuskaitytos variklių enkoderių pozicijos ir išsiunčiamos aukšto lygio sistemai. Šie duomenys vėliau gali būti naudojami odometrijos skaičiavimui ir navigacijos algoritmų darbui užtikrinti.

2.3. Aukšto lygio sistemos integravimas

Paruošus žemo lygio valdiklį ir mechaninę dalį, sistemos kūrimas buvo tęsiamas pagrindiniame kompiuteryje – Raspberry Pi 3.

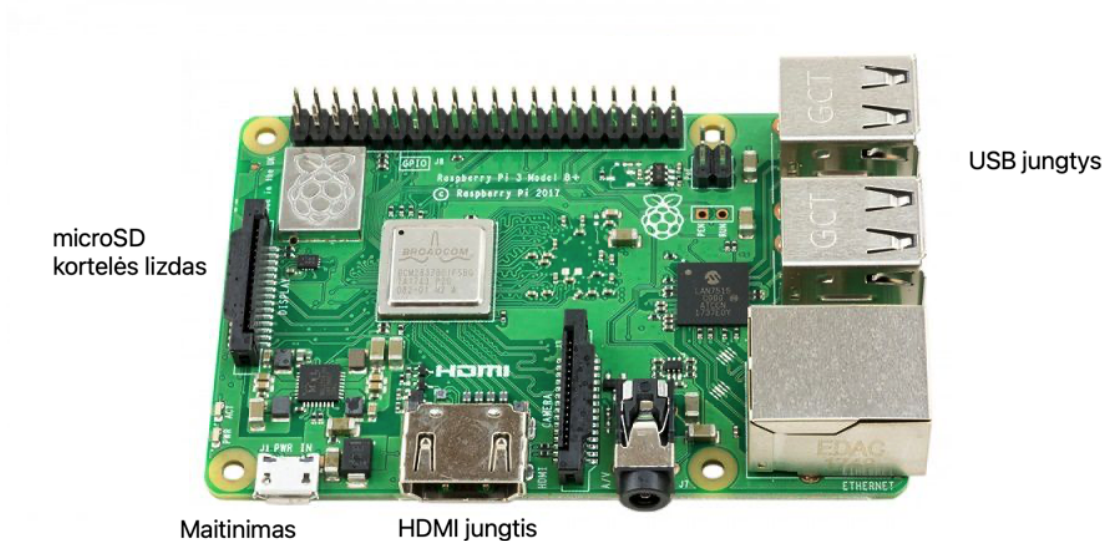
Šiame skyriuje aprašomas aukšto lygio valdymo sistemos integravimo procesas, kurio metu žemo lygio roboto valdymo programinė įranga buvo sujungta su aukšto lygio navigacijos ir autonominio tyrinėjimo sprendimais.

Aukšto lygio sistema realizuota naudojant ROS aplinką, kuri leidžia struktūrizuoti roboto programinę įrangą į atskirus mazgus ir užtikrinti jų tarpusavio komunikaciją. Tokia architektūra suteikia galimybę efektyviai keistis duomenimis tarp žemo lygio valdymo, jutiklių, žemėlapių sudarymo ir autonominio tyrinėjimo komponentų.

Toliau šiame skyriuje aprašomi pagrindiniai aukšto lygio sistemos komponentai, jų tarpusavio sąveika ir integravimo sprendimai.

2.3.1. Operacinės sistemos paruošimas

Pagrindinis roboto skaičiavimo kompiuteris – Raspberry Pi 3, pavaizduotas 6 paveikslėlyje.



6 pav. Raspberry Pi 3

Programinės įrangos diegimas Programinės aplinkos paruošimas buvo atliekamas vadovaujantis oficialiomis „TurtleBot3“ ir „ROBOTIS“ e-vadovo instrukcijomis [ROBd]. Įdiegta Ubuntu Server 22.04 LTS operacinė sistema, o joje sudiegtas **ROS2 Humble** karkasas.

Užkrovus operacinę sistemą, kompiuteris buvo prijungtas prie vietinio Wi-Fi tinklo. Kadan- gi robotas neturi ekrano ir klaviatūros, visas tolesnis sistemos valdymas, kodo kėlimas ir procesų stebėjimas buvo atliekamas nuotoliniu būdu per **SSH** protokolą iš asmeninio kompiuterio.

2.3.2. Programiniai moduliai

Paruošus operacinę sistemą, į Raspberry Pi buvo įkeltas programinis kodas. ROS2 aplin- ke sistema sukonstruota mazgų (angl. nodes) principu – tai atskiri procesai, kurie bendrauja tarpusavyje siųsdami pranešimus į „temas“ (angl. topics).

Šiame darbe sukurta architektūra susideda iš kelių pagrindinių mazgų, užtikrinančių duo- menų srautą tarp techninės įrangos ir navigacijos algoritmų.

LiDAR jutiklio mazgas

Vienas svarbiausių sistemos elementų – LIDAR jutiklio duomenų apdorojimas. Jutiklis LDS-02 per USB sąsają siunčia nuolatinį, neapdorotų baitų srautą. Standartinės ROS priemonės šio srauto nesupranta, todėl buvo sukurta Python kalba parašyta tvarkyklė.

1. Programa nuskaityto srautą ir ieško specifinės baitų sekos – 0x54 0x2C. Tai žymi 47 baitų duomenų paketo pradžią.
2. Radus antraštę, nuskaitymi likę baitai. Juose užkoduota informacija apie atstumą iki kliū- ties.
3. Išskirti duomenys konvertuojami į standartinį ROS LaserScan formatą ir publikuojami į /scan temą, kurią vėliau naudoja žemėlapių sudarymo algoritmai.

Variklių valdymo mazgas

Šis mazgas atlieka tarpininko vaidmenį tarp aukšto lygio navigacijos ir žemo lygio OpenCR valdiklio. Jo veikimas vyksta į dvi kryptis:

1. **Prenumeravimas:** Mazgas klausosi /cmd_vel temos, kurioje navigacijos algoritmai skel- bia norimą roboto linijinį ir kampinį greitį. Šie duomenys konvertuojami į tekstinę koman- dą (pvz., V 10 10) ir siunčiami į mikrovaldiklį.
2. **Publikavimas:** Mazgas priima ODOM pranešimus iš mikrovaldiklio. Čia atliekamas skaičia- vimas – enkoderių „tikai“ konvertuojami į metrus ir publikuojama į /odom temą. Galutinį ir tikslų roboto pozicijos (x, y) bei orientacijos (θ) nustatymą žemėlapyje atlieka **SLAM Toolbox** paketas.

Šiame etape sistema buvo pilnai realizuota ir išbandyta realioje aplinkoje. Vėlesnėse darbo stadijose, siekiant palengvinti autonominio tyrinėjimo algoritmų testavimą ir užtikrinti palyginamumą su simuliaciniais rezultatais, buvo priimtas sprendimas suvienodinti realiojo roboto ir simuliacijos aplinkas. Tai leido tą patį algoritmą testuoti skirtingose realizacijose, išlaikant tas pačias aplinkos sąlygas.

3. Programinė architektūra

Autonominio tyrinėjimo sistema realizuota Python programavimo kalba, naudojant ROS 2 karkasą. Sistema suprojektuota modulinio principu, leidžiančiu lengvai keisti tyrinėjimo algoritmus ir navigacijos metodus. Šiame skyriuje aprašoma programinės įrangos struktūra ir pagrindinių komponentų sąveika.

3.1. Sistemos struktūra

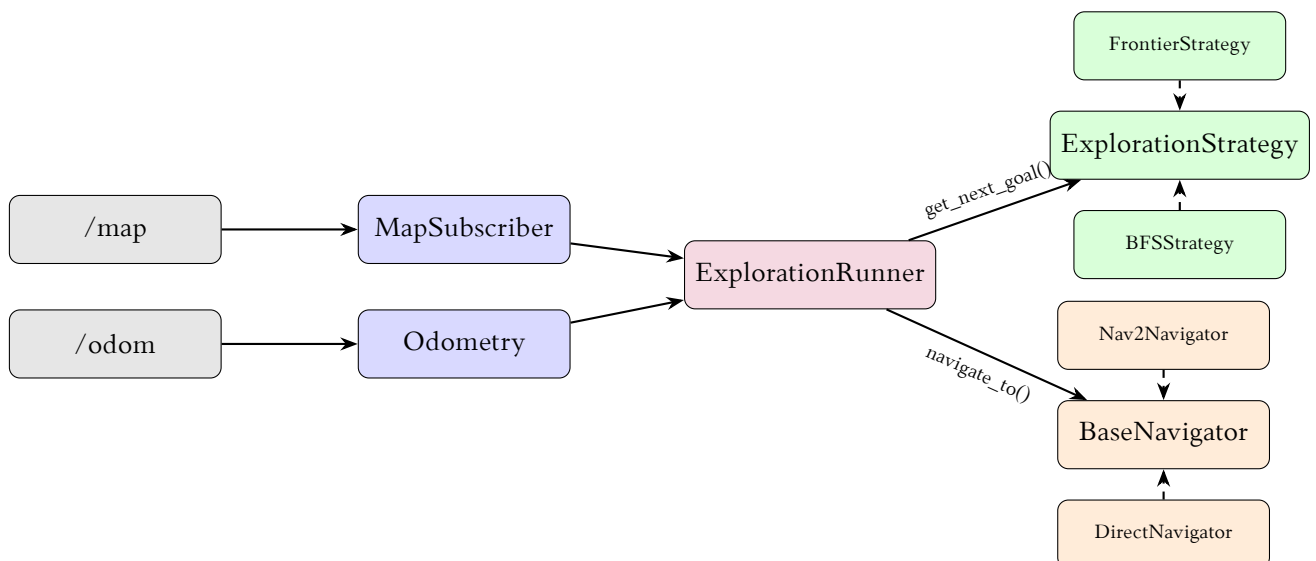
Programinis kodas organizuotas į keturis pagrindinius modulius, kiekvienas atsakingas už skirtingą funkcionalumo sritį:

- **core** – bazinė infrastruktūra: koordinačių transformacijos, žemėlapių duomenų gavimas, odometrija ir vizualizacija
- **algorithms** – tyrinėjimo algoritmai (Frontier, BFS), realizuojantys bendrą `ExplorationStrategy` sąsają
- **navigators** – navigacijos realizacijos: `Nav2Navigator` (naudoja Nav2 steką) ir `DirectNavigator` (tiesioginis valdymas)
- **runners** – tyrinėjimo ciklo orkestravimas, jungiantis visus komponentus

Tokia struktūra leidžia lengvai pridėti naujus tyrinėjimo algoritmus ar navigatorius, nekeičiant kitų sistemos dalių.

3.2. Architektūros diagrama

Sistemos komponentų sąveika pavaizduota 7 paveikslėlyje. Diagrama parodo duomenų srautą nuo ROS 2 temų iki roboto valdymo komandų.



7 pav. Autonominio tyrinėjimo sistemos architektūra

3.3. Pagrindinės klasės

Sistema naudoja objektinio programavimo principus, ypač strategijos ir kompozicijos projektavimo šablonus. Toliau aprašomos svarbiausios klasės.

3.3.1. ExplorationRunner

Pagrindinė tyrinėjimo orkestravimo klasė, kuri valdo visą tyrinėjimo ciklą. Ji paveldi `rcipy.node.Node` klasę ir veikia kaip ROS 2 mazgas. Kiekviename cikle:

1. Gauna naujausius žemėlapiu duomenis per `MapSubscriber`
2. Nuskaito roboto poziciją per `Odometry`
3. Iškviečia strategijos `get_next_goal()` metodą gauti kitą tikslą
4. Perduoda tikslą navigatoriui per `navigate_to()` metodą
5. Praneša strategijai apie rezultatą (`on_goal_reached` arba `on_goal_failed`)

3.3.2. ExplorationStrategy

Abstrakti bazinė klasė, apibrėžianti tyrinėjimo algoritmo sąsają. Kiekviena konkreti strategija turi realizuoti:

- `get_next_goal(grid, metadata, robot_position, path_planner)` – grąžina kitą tyrinėjimo tikslą arba `None`, jei tyrinėjimas baigtas
- `on_goal_reached(goal)` – iškviečiama pasiekus tikslą
- `on_goal_failed(goal)` – iškviečiama nepavykus pasiekti tikslo

Ši sąsaja leidžia lengvai pridėti naujus algoritmus – pakanka sukurti naują klasę, realizuojančią šiuos metodus.

3.3.3. BaseNavigator

Abstrakti navigatoriaus sąsaja, apibrėžianti kelio planavimo ir vykdymo metodus:

- `plan_path(target)` – suplanuoja kelią iki nurodyto taško, grąžina `PlannedPath` arba `None`
- `execute(path)` – įvykdo suplanuotą kelią, grąžina `NavigationResult`
- `navigate_to(target)` – patogus metodas, apjungiantis planavimą ir vykdymą

Sistemoje realizuoti du navigatoriai:

- **Nav2Navigator** – naudoja Nav2 navigacijos steką kelių planavimui ir vykdymui
- **DirectNavigator** – naudoja BFS kelių planavimą ir tiesioginį `/cmd_vel` valdymą

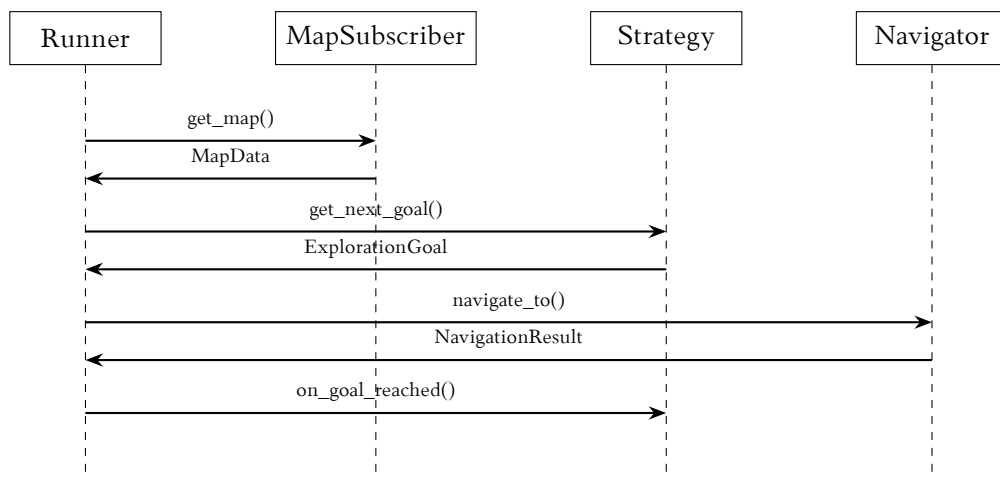
3.4. Duomenų struktūros

Sistema naudoja kelias pagrindines duomenų struktūras, realizuotas kaip Python `dataclass`:

- **Coords** – nekintamos 2D koordinatės pasaulio sistemoje (metrai), su atstumo skaičiavimo metodais
- **MapMetadata** – žemėlapių metaduomenys: skiriamoji geba, pradžios taškas, matmenys. Teikia koordinatinių transformacijų metodus `grid_to_world()` ir `world_to_grid()`
- **MapData** – užimtumo tinklas (numpy masyvas) kartu su metaduomenimis
- **ExplorationGoal** – tyrinėjimo tikslas su koordinatėmis, prioritetu ir papildomais strategijos duomenimis
- **PlannedPath** – suplanuotas kelias su tikslu, galutiniu tašku ir ilgiu
- **NavigationResult** – navigacijos rezultatas: sėkmės požymis ir nuvažiuotas atstumas

3.5. Tyrinėjimo ciklas

Visą tyrinėjimo procesą galima apibendrinti sekos diagrama, parodyta 8 paveikslėlyje.



8 pav. Tyrinėjimo ciklo sekos diagrama

Ciklas kartojamas tol, kol strategija grąžina `None` (nebėra neištirtų zonų) arba vartotojas nutraukia programą.

4. Simuliacinės aplinkos realizacija

Šiame skyriuje aprašoma simuliacinė aplinka, roboto modelio pritaikymas ir sukurti patalpos modeliai, naudojami algoritmų testavimui.

4.1. Roboto modelio pritaikymas

Standartinis TurtleBot3 Burger modelis Gazebo simulatoriuje naudoja LDS-01 LiDAR jutiklį, kurio veikimo diapazonas (0,12–3,5 m) skiriasi nuo fiziniame robote įdiegto LDS-02 jutiklio (0,16–8,0 m). Siekiant užtikrinti, kad simuliacijos rezultatai būtų tiesiogiai palyginami su fizinio roboto duomenimis, buvo modifikuoti roboto modelio parametrai.

Modifikacijos apėmė:

- **LiDAR diapazono keitimas:** Simuliacinio roboto LiDAR jutiklio minimalus ir maksimalus veikimo atstumas pakeisti į 0,16–8,0 m, atitinkančius LDS-02 specifikacijas.
- **SLAM parametrų derinimas:** SLAM Toolbox konfigūracijoje nustatyti `min_laser_range: 0.16` ir `max_laser_range: 8.0` parametrai.
- **Nav2 parametrų derinimas:** Navigacijos sistemos costmap konfigūracijose atitinkamai pakeisti `laser_min_range` ir `laser_max_range` parametrai.

Šie pakeitimai užtikrina, kad tas pats autonominio tyrinėjimo algoritmas veiktų identišškai tiek simuliacinėje, tiek realioje aplinkoje, o gauti rezultatai būtų tiesiogiai palyginami.

4.2. Patalpos modelio kūrimas

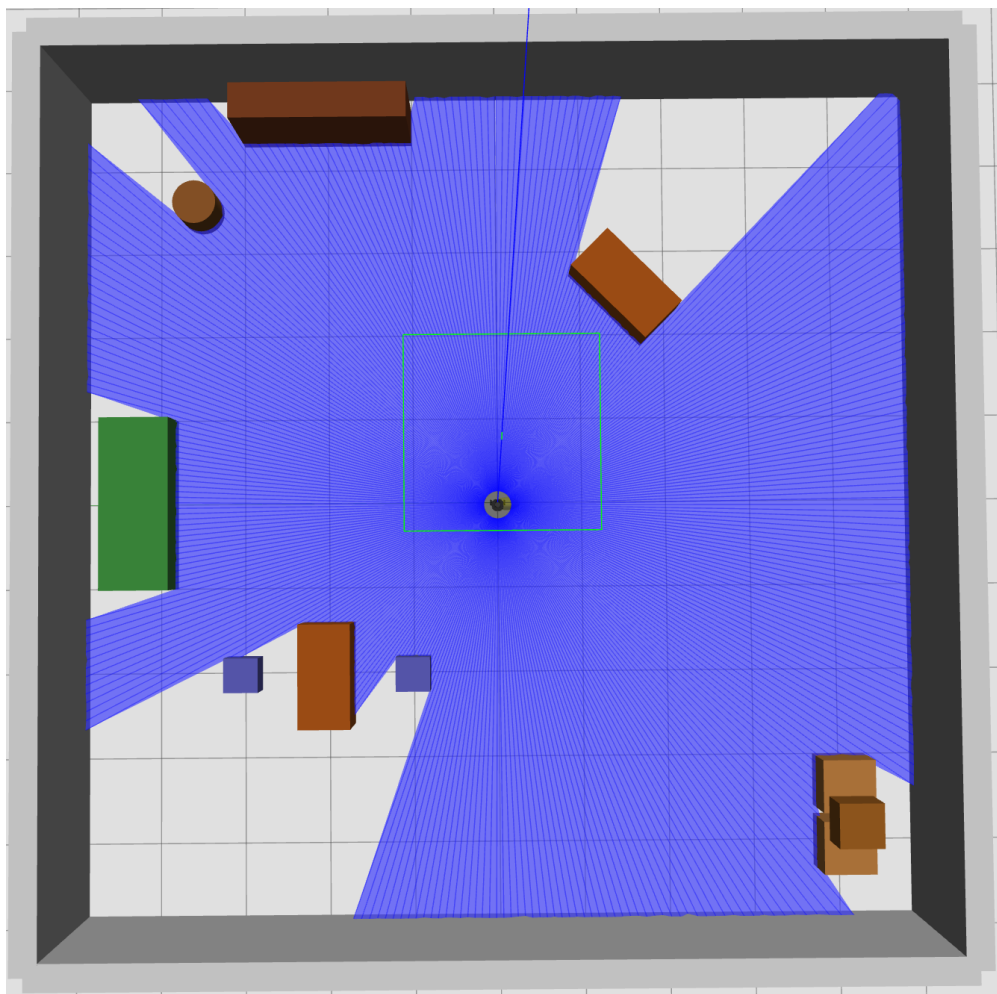
Eksperimentams sukurtas patalpos modelis Gazebo aplinkoje, leidžiantis įvertinti autonominio tyrinėjimo algoritmo veikimą. Modelis aprašytas SDF (Simulation Description Format) formatu, kuris leidžia tiksliai apibrėžti geometriją ir fizikos savybes. Visi objektai modelyje yra paprastos geometrinės figūros (stačiakampiai gretasieniai ir cilindrai), imituojančios realias kliūtis.

4.2.1. Simuliacinė patalpa

Patalpos modelis (`medium_room.world`) atitinka 10x10 m patalpą su išsibarsčiusiomis kliūtimis. Patalpos struktūra:

- Keturios išorinės sienos, sudarančios uždarą kvadratinę erdvę
- Įvairaus dydžio stačiakampės kliūtys, išdėstytos skirtingose patalpos vietose
- Cilindrinė kliūtis patalpos kampe

Šis modelis skirtas baziniam algoritmo testavimui atviroje erdvėje, kur robotas turi aptikti ir apeiti pavienius objektus. Patalpos išdėstymas pavaizduotas 9 paveikslėlyje.



9 pav. Simuliacinės patalpos modelis Gazebo aplinkoje (10x10 m)

5. Autonominio tyrinėjimo algoritmų realizacija

Šiame skyriuje aprašomas autonominio patalpų tyrinėjimo algoritmų realizavimo procesas, taikant ribomis paremtą (Nearest Frontier) ir BFS metodus.

Realizacijos metu pagrindinis dėmesys buvo skiriamas algoritmo integravimui su žemėlapių sudarymo (SLAM) sistema, navigacijos komponentais ir roboto judėjimo valdymu. Toks sprendimas leido užtikrinti nuoseklų autonominio tyrinėjimo procesą ir sudarė sąlygas vėlesniems eksperimentiniams bandymams bei rezultatų palyginimui skirtingose aplinkose.

Toliau šiame skyriuje atskirai nagrinėjama ribomis paremto ir BFS algoritmų realizacija.

5.1. Ribomis paremto algoritmo realizacija

Ribomis paremtas (angl. Frontier-based) tyrinėjimo algoritmas buvo realizuotas kaip autonominė strategija, veikianti kartu su Nav2 navigacijos sistema. Algoritmo pagrindas – ribų (frontierų) aptikimas naudojant morfologines operacijas ir sujungtų komponentų žymėjimą. Bendras algoritmo veikimo ciklas pateikiamas pseudokodu algoritme 1 algoritmas.

1 algoritmas Ribomis paremto tyrinėjimo algoritmas

```
1: INICIALIZACIJA:
2: Prenumeruoja: /map, /odom
3: Naudoja: Nav2 navigacijos sistemą
4: Būsena: blacklist  $\leftarrow$  []
5:
6: Funkcija exploration_cycle()
7: frontiers  $\leftarrow$  find_frontiers(map)
8: if frontiers =  $\emptyset$  then
9:   return Exploration_Complete
10: end if
11:
12: # Filtruoti pagal juodąjį sąrašą
13: valid_frontiers  $\leftarrow$  []
14: for frontier  $\in$  frontiers do
15:   if frontier  $\notin$  blacklist then
16:     valid_frontiers.append(frontier)
17:   end if
18: end for
19:
20: # Rasti geriausią frontiera
21: best_frontier  $\leftarrow$  None
22: best_path_length  $\leftarrow$   $\infty$ 
23: for frontier  $\in$  valid_frontiers do
24:   target  $\leftarrow$  frontier.get_representative_point()
25:   path  $\leftarrow$  Nav2.plan_path(target)
26:   if path  $\neq$  None ir path.length < best_path_length then
27:     best_frontier  $\leftarrow$  frontier
28:     best_path_length  $\leftarrow$  path.length
29:   end if
30: end for
31:
32: if best_frontier = None then
33:   return Exploration_Complete
34: end if
35:
36: result  $\leftarrow$  Nav2.navigate_to(best_frontier)
37: if result = Failed then
38:   blacklist.append(best_frontier)
39: end if
```

Sistema veikia cikliška – kiekvieno ciklo metu aptinkamos naujos ribos, pasirenkama artimiausia ir robotas nukreipiamas jos link. Žemiau detalizuojami pagrindiniai algoritmo moduliai.

5.1.1. Ribų aptikimas

Ribų aptikimui naudojamos morfologinės operacijos, leidžiančios efektyviai identifikuoti ribas tarp ištirtų ir neištirtų zonų. Aptikimo procesas vykdomas trimis etapais:

1. **Ribinių langelių identifikavimas:** Naudojant binarinę dilataciją, randami laisvi langeliai (reikšmė 0), kurie yra greta nežinomų langelių (reikšmė -1).
2. **Sienų kaimynystės filtravimas:** Pašalinami ribiniai langeliai, esantys šalia užimtų langelių (reikšmė 100), kad robotas nebūtų nukreiptas per arti sienų.
3. **Grupavimas į ribas:** Naudojant sujungtų komponentų žymėjimą (angl. Connected Component Labeling), gretimi ribiniai langeliai sujungiami į atskiras ribas.

Ribų aptikimo funkcijos pseudokodas pateikiamas algoritme 2 algoritmas.

2 algoritmas Ribų aptikimo funkcija

```
1: Funkcija find_frontiers(grid)
2:
3: # 1. Rasti ribinius langelius
4: free_cells ← grid = 0
5: unknown_cells ← grid = -1
6: frontier_cells ← free_cells ∧ dilate(unknown_cells)
7:
8: # 2. Pašalinti langelius šalia sienų
9: occupied_cells ← grid = 100
10: frontier_cells ← frontier_cells ∧ ¬dilate(occupied_cells)
11:
12: # 3. Sugrupuoti į ribas
13: dilated ← dilate(frontier_cells, iterations=2)
14: labeled, n ← connected_components(dilated)
15: labeled ← labeled × frontier_cells
16:
17: frontiers ← []
18: for  $i \leftarrow 1$  iki  $n$  do
19:   points ← langeliai, kur labeled =  $i$ 
20:   frontiers.append(Frontier(points))
21: end for
22:
23: return frontiers surūšiuotos pagal dydį mažėjančia tvarka
```

5.1.2. Ribos reprezentacinio taško parinkimas

Kiekvienai aptiktai ribai reikalingas vienas taškas, į kurį bus nukreiptas robotas. Tam naudojamas metodas, kuris randa artimiausią ribos langelį nuo jos geometrinio centro (centroido):

$$\text{centroid} = \left(\frac{1}{n} \sum_{i=1}^n r_i, \frac{1}{n} \sum_{i=1}^n c_i \right) \quad (1)$$

kur r_i ir c_i yra i -tojo ribos langelio eilutės ir stulpelio koordinatės, o n – bendras ribos langelių skaičius. Reprezentaciniu tašku pasirenkamas tas ribos langelis, kuris yra arčiausiai apskaičiuoto centroido.

5.1.3. Nepasiekiamų tikslų valdymas

Algoritmas palaiko juodąjį sąrašą (angl. Blacklist), į kurį įtraukiamos ribos, kurių nepavyko pasiekti. Riba įtraukiama į sąrašą, kai:

- Nav2 navigacijos sistema grąžina nesėkmės būseną.
- Robotas nepajudėjo link tikslo (užstrigo).
- Nepavyko suplanuoti kelio iki ribos.

Tikrinant ar riba priklauso juodajam sąrašui, naudojamas atstumas – jei ribos koordinatės yra arčiau nei 0,5 m nuo bet kurio įrašo sąrašė, ji laikoma užblokuota.

5.2. BFS algoritmo realizacija

Fizinio roboto valdymui realiu laiku buvo sukurtas artimiausio fronto paieškos algoritmas su kainos įvertinimu (angl. Cost-Aware BFS Frontier Exploration). Skirtingai nuo bazinių algoritmų, kurie orientuojasi tik į trumpiausią geometrinį atstumą, ši sistema priimdama sprendimus vertina ir judėjimo saugumą, pritaikydama baudos balus maršrutams, einantiems arti kliūčių. Bendras BFS pagrindu veikiančio tyrinėjimo algoritmo veikimo ciklas pateikiamas pseudokodu algoritme: 3 algoritmas.

3 algoritmas BFS išžvalgymo algoritmas

```
1: INICIALIZACIJA:
2: Prenumeruoja: /map, /odom, /scan
3: Publikuoja: /cmd_vel
4: Būsena: active_path ← [], failed_targets ← []
5:
6: Funkcija control_loop() # 5 Hz
7: if recovery_mode = True then
8:   Važiuoti atgal → sukti → ištrinti active_path
9:   return
10: end if
11:
12: if min_laser_dist < EMERGENCY_STOP_DIST (0.155m) then
13:   Publikuoti cmd_vel ← STOP
14:   if užstrigęs > 2s then
15:     failed_targets.append(current_frontier)
16:     recovery_mode ← True
17:   end if
18:   return
19: end if
20:
21: if active_path = [] then
22:   active_path ← get_path_to_frontier()
23:   if active_path = [] then
24:     Publikuoti angular.z ← 0.3 # Suktis vietoje
25:     return
26:   end if
27: end if
28:
29: # Sekti kelią
30: target ← active_path[0]
31: Apskaičiuoti distance, angle_error iki target
32: if distance < WAYPOINT_TOLERANCE (0.15m) then
33:   active_path.pop(0)
34: end if
35: if |angle_error| > 0.4 rad then
36:   Publikuoti tik angular.z
37: else
38:   Publikuoti linear.x ir angular.z
39: end if
```

Programa veikia cikliška 5 Hz dažniu (kas 0.2 sekundės), nuolat apdorojama LIDAR jutiklio, odometrijos ir žemėlapyio duomenis. Žemiau detalizuojami trys pagrindiniai sistemos moduliai: maršruto planavimas, trajektorijos sekimas ir saugumo užtikrinimas.

5.2.1. Maršruto planavimas

Algoritmo pagrindas yra modifikuotas paieškos pločiu algoritmas (BFS), kuris paleidžiamas, kai robotas neturi aktyvaus maršruto. Šios funkcijos pseudokodas pateikiamas algoritme: 4 algoritmas. Paieškos tikslas – rasti ribą: tinklo langelį, kurio reikšmė žemėlapyje yra nežinoma (-1) ir kuri nėra įtraukta į nepasiekiamų langelių sąrašą (angl. Blacklist).

4 algoritmas Pigiausio kelio radimo iki ribos funkcija

```

1: Funkcija get_path_to_frontier()
2: heapq  $\leftarrow [(0, \text{start\_node})]$ 
3: came_from  $\leftarrow \{\}$ , cost_so_far  $\leftarrow \{\}$ 
4: found_frontiers  $\leftarrow []$ 
5:
6: while heapq  $\neq \emptyset$  ir iteracijos  $< 10000$  do
7:   current_cost, current  $\leftarrow \text{heappop}(\text{heapq})$ 
8:
9:   if map_data[current] = -1 ir current  $\notin \text{failed\_targets}$  then
10:     Atkurti path iš came_from
11:     found_frontiers.append((position, path, current_cost))
12:   end if
13:
14:   for neighbor  $\in [(0,1), (0, -1), (1,0), (-1,0)]$  do
15:     if is_safe_cell(neighbor) then
16:       wall_dist  $\leftarrow \text{get\_distance\_to\_wall}(\text{neighbor})$ 
17:       wall_penalty  $\leftarrow (4 - \text{wall\_dist}) \times 5$ 
18:       new_cost  $\leftarrow \text{cost\_so\_far}[\text{current}] + 1 + \text{wall\_penalty}$ 
19:       heappush(heapq, (new_cost, neighbor))
20:     end if
21:   end for
22: end while
23:
24: if found_frontiers  $\neq []$  then
25:   Surūšiuoti found_frontiers pagal kainą
26:   return pigiausias kelias
27: else
28:   return []
29: end if

```

Siekiant, kad robotas vengtų sienų, paieškai naudojama prioritetinga eilė (angl. Priority Queue). Kiekvienai tikrinamai ląstelei apskaičiuojama judėjimo kaina:

$$C_{\text{viso}} = C_{\text{atstumas}} + C_{\text{bauda}}$$

- C_{atstumas} – žingsnių skaičius nuo roboto iki langelio.
- C_{bauda} – papildoma vertė, priklausanti nuo atstumo iki artimiausios kliūtis. Jei langelis yra arti sienos, jam priskiriama aukšta kaina.

Algoritmas pirmiausia nagrinėja „pigiausias“ (saugiausias) ląsteles. Tai lemia, kad sugeneruotas kelias natūraliai vingiuoja patalpos centru, net jei geometriškai tiesesnis kelias būtų palei sieną. Maršruto apdorojimas Suradus pasiekiamus frontus, jie išrikiuojami pagal kainą ir pasirenkamas optimalus. Gautas neapdorotas kelias yra optimizuojamas:

1. **Filtravimas:** Taškai esantys per arti roboto (<20cm) yra atmetami, kad būtų išvengta nereikalingų manevrų.
2. **Retinimas:** Paliekamas tik kas trečias kelio taškas. Tai sumažina skaičiavimų kiekį bei roboto judėjimą padaro sklandesnį.

5.2.2. Trajektorijos sekimas

Sėkmingai sugeneravus maršrutą yra vykdomas kelio sekimas. Jis lygina dabartinę roboto orientaciją su kryptimi į artimiausią tašką ir rašo greičio komandas. Siekiant tikslumo, judėjimo logika suskirstyta į dvi fazes priklausomai nuo kampinės paklaidos:

1. Jei kampinė paklaida viršija 23° , robotas sustabdo linijinį judėjimą ir sukasi vietoje pastoviu greičiu.
2. Kai paklaida maža, robotas juda į priekį, o posūkio greitis reguliuojamas atitinkamai.

Tarpinis taškas laikomas pasiektu, kai atstumas iki jo tampa mažesnis nei 15 cm. Tuomet tarpinis taškas iš maršruto sąrašo yra pašalinamas ir imamas kitas taškas.

5.2.3. Saugumo mechanizmai

Kadangi darbo metu buvo susidurta su tokiais problemomis kaip: kliūčių kliudimas ar užstrigimas bandant pasiekti tam tikrą tašką sistemoje buvo implementuoti saugumo mechanizmai. Šioms problemoms spręsti buvo pasirinkti šie sprendimai:

Avarinis stabdymas ir juodasis sąrašas

Kiekvieno ciklo metu tikrinami LIDAR duomenys. Jei fiksuojama kliūtis arčiau nei 15.5 cm, aktyvuojamas avarinis stabdymas. Jei robotas stovi vietoje ilgiau nei 2 sekundes (negali pasiekti tikslo), dabartinis tikslas įtraukiamas į juodąjį sąrašą (angl. blacklist). Tai užtikrina, kad algoritmas nebandytų be galo važiuoti į tą patį nepasiekiamą tašką ir ieškotų alternatyvų.

Atsistatymo režimas

Jei robotas užstringa arba negali rasti kelio, inicijuojamas automatinis atsistatymo manevras, susidedantis iš dviejų etapų:

1. **Persiorientavimas:** Robotas lėtai važiuoja atbulas. Tai padeda atsitraukti nuo kliūčių.
2. **Skenavimas:** Atliekamas lėtas apsisukimas vietoje. Tai leidžia atnaujinti žemėlapi ir pamatyti naujus praėjimus, kurie anksčiau buvo užstoti.

Ši kompleksinė logika – kelių kainos vertinimas, tikslus valdymas ir autonominis atsistatymas – leidžia robotui sėkmingai ištirti patalpą be operatoriaus įsikišimo.

Rezultatai ir išvados

Šiame skyriuje pateikiami autonominio patalpų tyrinėjimo bandymų rezultatai, gauti taikant ribomis paremtą ir BFS algoritmus realioje bei simuliacinėje aplinkoje. Skyriaus pabaigoje pateikiamos darbo išvados, apibendrinančios gautus rezultatus ir pasirinktų metodų taikymo patirtį.

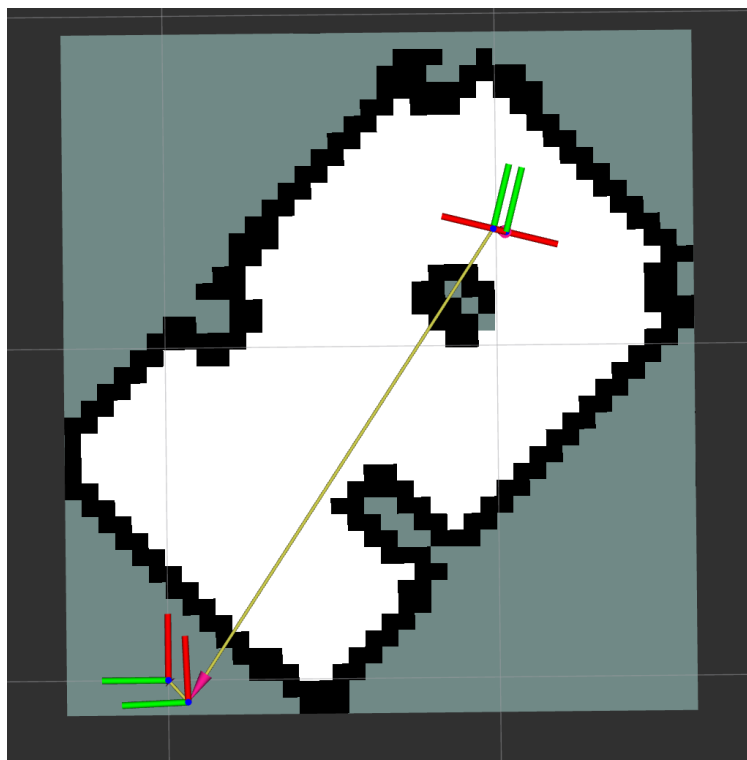
Rezultatai fizinėje aplinkoje

Autonominio tyrinėjimo algoritmai buvo testuojami realioje patalpų aplinkoje, esant panašioms pradinėms sąlygoms. Kiekvienas algoritmas buvo paleistas kelis kartus, o pateikiami rezultatai atspindi gautų matavimų vidurkį.

Ribomis paremto algoritmo rezultatai

Taikant ribomis paremtą algoritmą, patalpos tyrinėjimas vidutiniškai truko **60,1 s**, per kurio robotas nuvažiavo **0,32 m** ir padengė **1,52 m²** plotą. Gautos skaitinės reikšmės atspindi kelių bandymų vidurkį. Vizualinei analizei buvo pasirinktas vienas reprezentatyvus bandymas, kurio metu sudarytas žemėlapis geriausiai atspindi tipinį algoritmo veikimą realioje aplinkoje.

Sudarytas žemėlapis pasižymėjo tvarkinga ir nuoseklia struktūra – kliūtys ir sienos buvo aiškiai bei tolygiai pažymėtos, ypač palei patalpos kraštus. Nebuvo pastebėta ryškių spragų ar neištirtų zonų, o tai rodo stabilų algoritmo veikimą tyrinėjimo metu. Ribomis paremto algoritmo sudarytas žemėlapis pateiktas 10 paveiksle.

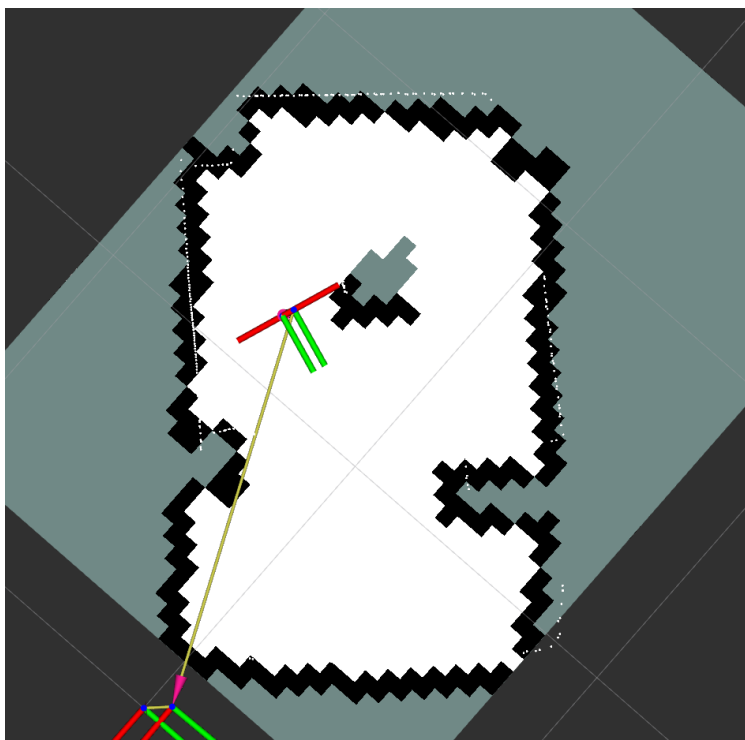


10 pav. Patalpos žemėlapis sudarytas naudojant ribomis paremtą algoritmą (realus robotas)

BFS algoritmo rezultatai

Naudojant BFS algoritmą, tyrinėjimo trukmė vidutiniškai siekė **85,2 s**, nuvažiuotas atstumas – **1,06 m**, o padengtas plotas – **1,59 m²**. Nors pateikiami skaitiniai rezultatai atspindi kelių bandymų vidurkį, vizualinei analizei buvo pasirinktas vienas reprezentatyvus bandymas, kurio metu gautas žemėlapis geriausiai atspindi tipinį algoritmo elgesį realioje aplinkoje.

Sudarytas žemėlapis buvo panašus į gautą ribomis paremto metodo atveju, tačiau vienoje vietoje pastebėta nedidelė sienelės fragmento spraga. BFS algoritmo sudarytas žemėlapis pateiktas 11 paveiksle.



11 pav. Patalpos žemėlapis sudarytas naudojant BFS algoritmą (realus robotas)

Rezultatų palyginimas

Lyginant abu algoritmus realioje aplinkoje, ribomis paremtas metodas pasižymėjo trumpesniu tyrinėjimo laiku ir mažesniu nuvažiuotu atstumu, tuo tarpu BFS algoritmas užtikrino kiek didesnę padengtą plotą ir stabilesnį roboto judėjimą.

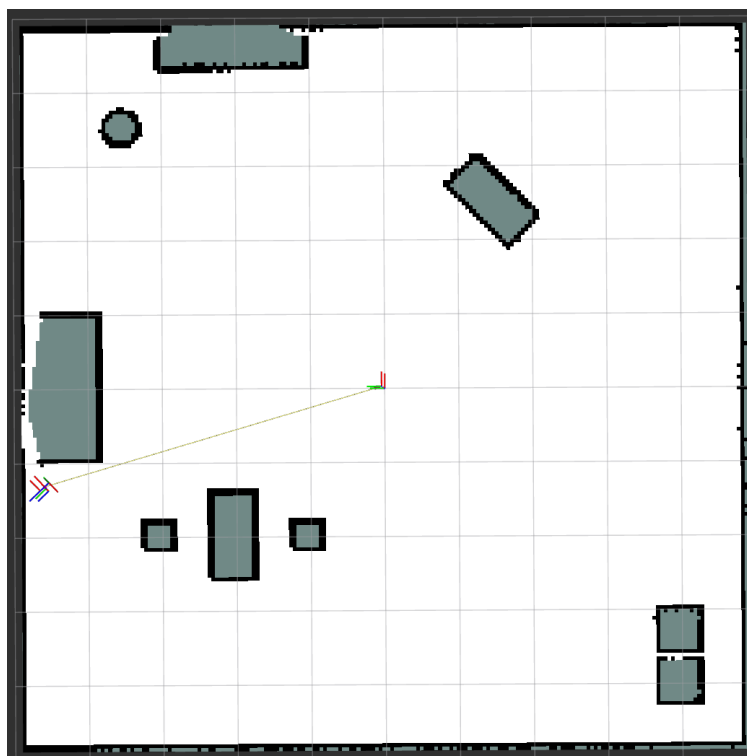
Rezultatai simuliacinėje aplinkoje

Simuliacinėje aplinkoje abu algoritmai buvo testuojami identiškomis sąlygomis, naudojant Gazebo simulatorių su vidutiniu sudėtingumo patalpos modeliu. Simuliacija leido atlikti ilgesnius tyrinėjimo bandymus ir tiksliau įvertinti algoritmų elgseną didesnėse erdvėse.

Ribomis paremto algoritmo rezultatai

Taikant ribomis paremtą algoritmą simuliacinėje aplinkoje, tyrinėjimas truko **372,6 s**, per kuriuos robotas nuvažiavo **38,7 m** ir padengė **86,7 m²** plotą. Algoritmas sėkmingai ištyrinėjo visą simuliacinę patalpą, sudarydamas išsamų žemėlapią su aiškiai pažymėtomis sienomis ir kliūtimis.

Sudarytas žemėlapis pasižymėjo aukšta kokybe – visos patalpos sienos buvo tolygiai ir tiksliai atvaizduotos, nebuvo pastebėta reikšmingų spragų ar neištirtų zonų. Ribomis paremto algoritmo sudarytas žemėlapis simuliacinėje aplinkoje pateiktas 12 paveiksle.

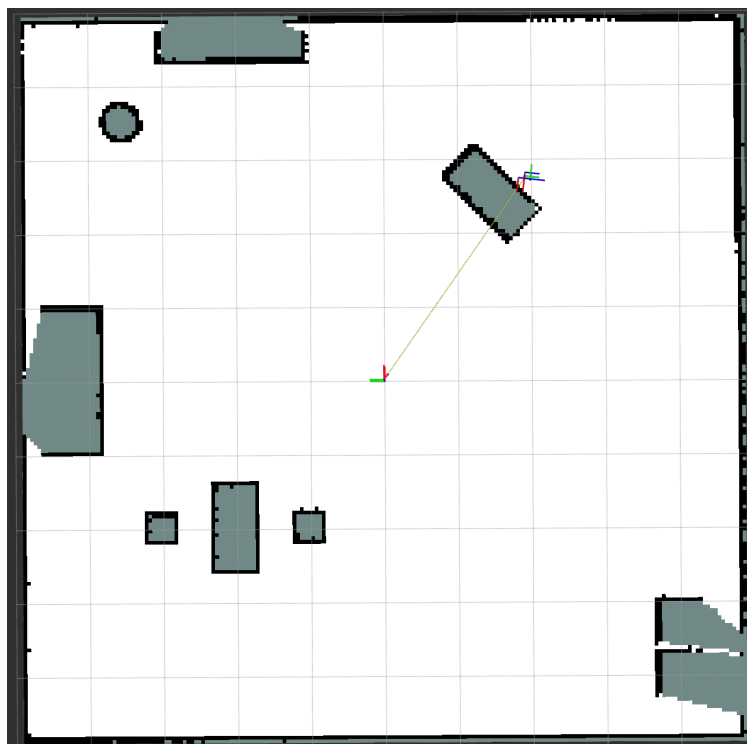


12 pav. Patalpos žemėlapis sudarytas naudojant ribomis paremtą algoritmą (simuliacija)

BFS algoritmo rezultatai

Naudojant BFS algoritmą, tyrinėjimo trukmė siekė **210,2 s**, nuvažiuotas atstumas – **21,8 m**, o padengtas plotas – **86,1 m²**. BFS algoritmas parodė greitesnį tyrinėjimo tempą, tačiau nuvažiavo mažesnę bendrą atstumą nei ribomis paremtas metodas.

Sudarytas žemėlapis buvo panašios kokybės kaip ir ribomis paremto algoritmo atveju, su aiškiai pažymėtomis sienomis ir kliūtimis. BFS algoritmo sudarytas žemėlapis simuliacinėje aplinkoje pateiktas 13 paveiksle.



13 pav. Patalpos žemėlapis sudarytas naudojant BFS algoritmą (simuliacija)

Rezultatų palyginimas

Simuliacinėje aplinkoje BFS algoritmas pasižymėjo trumpesniu tyrinėjimo laiku (210 s prieš 373 s) ir efektyvesniu judėjimu – padengė panašų plotą nuvažiavęs beveik dvigubai mažesnę atstumą. Tai rodo, kad BFS algoritmas renka optimalesnį maršrutą, vengdamas nereikalingų grįžimų. Tuo tarpu ribomis paremtas algoritmas, nors ir lėtesnis, užtikrina išsamesnį patalpos padengimą su mažiau neištirtų zonų.

Išvados

1. Atlikta literatūros analizė leido apibrėžti autonominio patalpų tyrinėjimo ir SLAM sprendimų taikymo principus.
2. Paruošta ir sukonfigūruota fizinė roboto sistema, užtikrinanti patikimą jutiklių ir pavarų veikimą realioje aplinkoje.
3. Sukurta simuliacinė aplinka sudarė sąlygas testuoti algoritmus ir palyginti juos su realiais bandymais.
4. Darbo metu įgyvendinti ribomis paremtas bei kainomis paremtas BFS algoritmai, leidę autonomiškai tirti aplinką ir sudaryti jos žemėlapi.
5. Eksperimentai parodė, kad ribomis paremtas metodas veikia greičiau, o kainomis paremtas BFS algoritmas užtikrina stabilesnį ir saugesnį judėjimą.

6. Nustatyta, kad realioje aplinkoje algoritmų veikimą reikšmingai veikia fiziniai ir jutiklių apribojimai, neatsispindintys simuliacijoje.

Literatūra ir šaltiniai

- [Ard] Arduino. *Arduino Integrated Development Environment*. url: <https://docs.arduino.cc/software/ide-v1/tutorials/arduino-ide-v1-basics/> (žiūrėta 2025-10).
- [AS19] R. Amsters, P. Slaets. „Turtlebot 3 as a robotics education platform“. Iš: *International Conference on Robotics in Education* (2019).
- [CC07] T. Collins, J. J. Collins. „Occupancy grid mapping: An empirical evaluation“. Iš: *2007 mediterranean conference on control automation*. (2007).
- [CLR⁺22] T. Cormen, C. Leiserson, R. Rivest, C. Stein. *Introduction to algorithms*. 594-597 p. MIT press, 2022.
- [Yam97] B. Yamauchi. „A frontier-based approach for autonomous exploration.“ Iš: *proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation* (1997).
- [M]21] S. Macenski, I. Jambrecic. „SLAM Toolbox: SLAM for the dynamic world.“ Iš: *Journal of Open Source Software*, 6(61), 2783. (2021).
- [Pi] R. Pi. *Raspberry Pi - Getting Started*. url: <https://www.raspberrypi.com/documentation/computers/getting-started.html> (žiūrėta 2025-10).
- [QCG⁺09] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A. Ng. „ROS: an open-source Robot Operating System“. Iš: *ICRA Workshop on Open Source Software* (2009).
- [ROBa] ROBOTIS. *DYNAMIXEL Wizard 2.0*. url: https://emanual.robotis.com/docs/en/software/dynamixel/dynamixel_wizard2/ (žiūrėta 2025-10).
- [ROBb] ROBOTIS. *LDS-02*. url: https://emanual.robotis.com/docs/en/platform/turtlebot3/appendix_lds_02/ (žiūrėta 2025-10).
- [ROBc] ROBOTIS. *OpenCR 1.0*. url: <https://emanual.robotis.com/docs/en/parts/controller/opencr10/> (žiūrėta 2025-10).
- [ROBd] ROBOTIS. *SBC setup*. url: https://emanual.robotis.com/docs/en/platform/turtlebot3/sbc_setup/#sbc-setup (žiūrėta 2025-10).
- [ROBe] ROBOTIS. *Turtlebot3*. url: <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/#overview> (žiūrėta 2025-10).
- [ROBf] ROBOTIS. *XM430-W350*. url: <https://emanual.robotis.com/docs/en/dxl/x/xm430-w350/> (žiūrėta 2025-10).